



HTOC

HEALTHCARE THREAT OPERATIONS CENTER

Next Observed Indicator Forecasting

Version 3.0
April 2026

Table of Contents

Table of Contents	4
1. Purpose	6
2. Scope	6
3. Prerequisites	6
3.1 Environment	6
3.2 Input Data Files	7
4. Key Configuration Parameters	7
5. Pipeline Overview	8
6. Step-by-Step Execution	9
Step 1 — Import dependencies	9
Step 2 — Set Date Window	9
Step 3 — Load Daily OpDiv CSVs	10
Step 4 — Clean and Reshape	10
Step 5 — Build Per-OpDiv Dense Panel	11
Step 6 — Define Modeling Functions	11
Step 7 — Run <code>main()</code>	12
7. Features Used in Modeling	13
7.1 Time-Series Features (computed per indicator)	13
7.2 Supervised Labels (computed per indicator per horizon)	14
8. Forecasting Models	14
8.1 Logistic Regression	14
8.2 Gradient Boosting Classifier	15
8.3 Exponential (Poisson-Rate) Model	15
8.4 Weibull AFT (Accelerated Failure Time) Model	15
8.5 Weighted Ensemble	15
9. Confidence Labels	16
10. Production Output Table	16
11. How to Run	17
11.1 Run the Python script	17
11.2 Verify prerequisites	18
11.3 Monitor progress	18

11.4 Enable file output (optional)	18
11.5 Typical run time	18
11.6 Scheduled execution	19
12. Troubleshooting	19
13. Optional: Feedback Loop and Model Retraining	20
How it works:	20
Notes:	21
14. Interpreting Results	21
15. Maintenance Notes	22
16. Appendix - Standalone Python Script	22
17. Related Documents	23

Table of Contents

Table of Contents	4
1. Purpose	5
2. Scope	5
3. Prerequisites	5
4. Key Configuration Parameters	6
5. Pipeline Overview	7
6. Step-by-Step Execution	8
7. Features Used in Modeling	12
8. Forecasting Models	13
9. Confidence Labels	15
10. Production Output Table	15
11. How to Run	16
12. Troubleshooting	18
13. Optional: Feedback Loop and Model Retraining	19
14. Interpreting Results	20
15. Maintenance Notes	21
16. Appendix - Standalone Python Script	21
17. Related Documents	22

Table of Contents 4

1. Purpose	6
2. Scope	6
3. Prerequisites	6
3.1 Environment	6
3.2 Input Data Files	7
4. Key Configuration Parameters	7
5. Pipeline Overview	8
6. Step-by-Step Execution	9
Step 1 — Import dependencies	9
Step 2 — Set Date Window	9
Step 3 — Load Daily OpDiv CSVs	10
Step 4 — Clean and Reshape	10

Step 5 — Build Per-OpDiv Dense Panel	11
Step 6 — Define Modeling Functions	11
Step 7 — Run <code>main()</code>	12
7. Features Used in Modeling	13
7.1 Time-Series Features (computed per indicator)	13
7.2 Supervised Labels (computed per indicator per horizon)	14
8. Forecasting Models	14
8.1 Logistic Regression	14
8.2 Gradient Boosting Classifier	15
8.3 Exponential (Poisson-Rate) Model	15
8.4 Weibull AFT (Accelerated Failure Time) Model	15
8.5 Weighted Ensemble	15
9. Confidence Labels	16
10. Production Output Table	16
11. How to Run	17
11.1 Run the Python script	17
11.2 Verify prerequisites	18
11.3 Monitor progress	18
11.4 Enable file output (optional)	18
11.5 Typical run time	18
11.6 Scheduled execution	19
12. Troubleshooting	19
13. Optional: Feedback Loop and Model Retraining	20
How it works:	20
Notes:	21
14. Interpreting Results	21
15. Maintenance Notes	22
16. Appendix - Standalone Python Script	22
17. Related Documents	23

1. Purpose

This SOP describes how to operate the Next Observed Indicator (NOI) forecasting pipeline. The script loads daily OpDiv observation records, engineers time-series features for each indicator per OpDiv, and runs a multi-model ensemble to forecast the probability that each indicator will be observed again within **1, 7, 14, 30, and 45-day** horizons. Results are presented as a production-ready prioritization table with categorical confidence labels.

-

2. Scope

This procedure applies to HTOC analysts and data engineers who run, maintain, or consume indicator observation forecasts. It covers end-to-end execution of the forecasting **Python script** (Appendix path and deployed copies), from environment setup through production output review. The optional feedback/retraining section is also documented for users who wish to improve model accuracy over time.

-

3. Prerequisites

3.1 Environment

Requirement	Notes
Python 3.x	Compatible with numpy, pandas, scikit-learn, lifelines

Requirement	Notes
scikit-learn	`LogisticRegression`, `GradientBoostingClassifier`, `StandardScaler`, `Pipeline`
lifelines	`WeibullAFTFitter` for survival-model probabilities
numpy, pandas	Standard data processing
Access to `Z:\HTOC\Data_Analytics\`	All input OpDiv CSV files live here

Install missing packages if needed:

```
pip install scikit-learn lifelines
```

3.2 Input Data Files

The pipeline loads daily OpDiv observation CSVs using the following path template:

```
Z:/HTOC/Data_Analytics/Data/OpDiv_Observations/htoc_opdiv_obs_d{YYYYMMDD}.csv
```

Files must be available for the **100 days prior to today**. If any daily files are missing, the loader skips them gracefully, but gaps will reduce feature quality for affected indicators.

-

4. Key Configuration Parameters

These values are defined near the top of the source. Review them before each run.

Parameter	Default	Description
`start_date` offset	100 days before today	How far back daily CSV files are loaded
`end_date` offset	0 days (today)	End of the observation window
`n_days` (in `load_data`)	100	Used by the standalone `load_data` helper (not called in the main pipeline)
`high_thresh`	0.60	Minimum ensemble probability for a "Highly likely" confidence label
Secondary probability threshold	0.07	Minimum probability for "Possibly active" label
Frequency threshold (High)	≥ 2	Minimum observation frequency count for "Highly likely" classification
Frequency threshold (Possibly active)	≥ 1	Minimum observation frequency count for "Possibly active" classification
Exponential rate floor	1e-6	Prevents division by zero in the Poisson-style model

-

5. Pipeline Overview

The pipeline executes in the following logical stages in a single Python process run.

[1] Import Dependencies
↓


```
[2] Set Date Window & File Path Template
    ↓
[3] Load & Concatenate Daily OpDiv CSVs
    ↓
[4] Clean & Reshape Data
    ↓
[5] Build Per-OpDiv Dense Panel (Cartesian Product)
    ↓
[6] Define Modeling Functions
    ↓
[7] Run main() – Feature Engineering + All Models + Ensemble
    ↓
[8] Display Production Output Table
    ↓
[9] (Optional) Feedback loop and model retraining stub
```

-

6. Step-by-Step Execution

Step 1 — Import dependencies

At startup the script imports:

```
numpy, pandas, os, datetime, warnings
```

```
sklearn: LogisticRegression, GradientBoostingClassifier, StandardScaler, Pipeline
```

```
lifelines: WeibullAFTFitter
```

Verify: No import errors. If `lifelines` is missing, install it before proceeding.

-

Step 2 — Set Date Window

The script calculates:

```
start_date = today - 100 days
```

```
end_date = today
```

A list of all dates in this range is generated (`datelist`) and printed. The file path template for OpDiv CSVs is also defined here.

Verify: The printed date list covers the expected 100-day window ending today.

-

Step 3 — Load Daily OpDiv CSVs

The `load_files()` function iterates over `datelist`, attempts to read each daily CSV at the defined path template, skips missing files silently, and concatenates all loaded files into a single dataframe `src`.

Verify: `src` is non-empty and contains the expected columns (`indicator`, `OpDiv`, `obs_date`, `observations`). If `src` is empty, check that the file path template and [Z:\](#) drive mapping are correct.

-

Step 4 — Clean and Reshape

Drops `curr_date` and `indicator_key` columns if present.

Renames `obs_date` → `date`.

Parses `date` as `datetime`.

Strips whitespace from `indicator` and `OpDiv` columns.

-

Step 5 — Build Per-OpDiv Dense Panel

For each OpDiv in the data, a **full Cartesian product** is created across:

All unique API users

All calendar days from the minimum date in data through today

All unique indicators

This dense grid is left-merged with actual observation counts. Missing days are filled with 0 observations. Additional calendar features are added:

Column	Description
`dayofweek`	Integer 0–6 (Monday = 0)
`is_weekend`	1 if Saturday or Sunday, else 0
`day`	Day of month
`month`	Month number
`seen`	Binary: 1 if `observations > 0`, else 0

The result is stored in `opdiv_merged`, a dictionary keyed by OpDiv name.

-

Step 6 — Define Modeling Functions

This portion of the script defines all helper functions used by `main()`. No modeling runs until `main()` is invoked. Key functions:

Function	Purpose
<code>`extract_time_series_features(group)`</code>	Computes per-indicator features from the <code>`seen`</code> series
<code>`build_features(df)`</code>	Applies <code>`extract_time_series_features`</code> across all indicators for an OpDiv
<code>`train_predict(X, y)`</code>	Trains and scores a Logistic Regression pipeline
<code>`train_gbt(X, y)`</code>	Trains and scores a Gradient Boosting Classifier
<code>`get_model_outputs(df)`</code>	Runs all four model types and merges results
<code>`add_rule_and_ensemble(df)`</code>	Applies rule-based labels and computes weighted ensemble
<code>`classify_window(prob, freq)`</code>	Returns confidence label string
<code>`add_confidence_and_format(df)`</code>	Formats probabilities as percentages and assigns confidence labels
<code>`build_production_output(df)`</code>	Selects and renames columns for the final output table

-

Step 7 — Run `main()`

The `main()` function is the core of the pipeline. It loops over each OpDiv in `opdiv_merged` and runs the full modeling sequence:

Calls `build_features()` to extract time-series features per indicator.

Calls `get_model_outputs()` to generate raw model probabilities.

Calls `add_rule_and_ensemble()` to apply rules and compute ensemble scores.

Calls `add_confidence_and_format()` to apply confidence thresholds and format output.

Calls `build_production_output()` to produce the final display table.

Results are stored in a dictionary (`outputs`) keyed by OpDiv name.

Prediction output path (often commented out in `main()`):

`C:\Users\jaskew\Documents\NOI_Logs\Predictions\`

Uncomment the CSV save block inside `main()` if persistent file output is needed.

-

7. Features Used in Modeling

7.1 Time-Series Features (computed per indicator)

Feature	Description
<code>`last_seen`</code>	Days since the indicator was last observed
<code>`freq_1`</code>	Observation count in the last 1 day
<code>`freq_7`</code>	Observation count in the last 7 days
<code>`freq_14`</code>	Observation count in the last 14 days
<code>`freq_30`</code>	Observation count in the last 30 days
<code>`freq_45`</code>	Observation count in the last 45 days
<code>`avg_gap`</code>	Average number of days between observations
<code>`burstiness`</code>	Coefficient of variation of inter-observation gaps (high = irregular)

7.2 Supervised Labels (computed per indicator per horizon)

Label	Description
`label_7`	1 if indicator was seen at least once in the last 7 days
`label_14`	1 if indicator was seen at least once in the last 14 days
`label_30`	1 if indicator was seen at least once in the last 30 days
`label_45`	Binary flag derived from the 45th-from-last row (see note below)

> **Note:** `label_45` uses `series[-45]` (a single index) rather than a slice `series[-45:]`. This means the 45-day label reflects only whether the indicator was seen on that specific day, not whether it was seen at all in the 45-day window. Be aware of this when interpreting 45-day probabilities.

-

8. Forecasting Models

The pipeline uses four models, combined into a weighted ensemble.

8.1 Logistic Regression

Implementation: `sklearn.pipeline.Pipeline` with `StandardScaler` + `LogisticRegression`

Horizons: 7, 14, 30, 45 days (1-day reuses the 7-day label as its training target)

Features: All 8 time-series features

Guard: If only one class is present in training data, the model returns NaN probabilities (handled downstream)

8.2 Gradient Boosting Classifier

Implementation: `sklearn.ensemble.GradientBoostingClassifier` (default hyperparameters)

Horizons: 7, 14, 30, 45 days (same 1-day reuse as logistic)

Features: All 8 time-series features

Purpose: Captures non-linear patterns and feature interactions

8.3 Exponential (Poisson-Rate) Model

Formula: $P(\text{seen within } t \text{ days}) = 1 - \exp(-\text{rate} \times t)$

Rate: `freq_30 / 30`, clipped to a minimum of $1e-6$

Horizons: 1, 7, 14, 30, 45 days

Purpose: Models indicators as a memoryless Poisson process; good for regularly-seen indicators

8.4 Weibull AFT (Accelerated Failure Time) Model

Implementation: `lifelines.WeibullAFTFitter`

Duration variable: `avg_gap`

Event variable: `label_7` (7-day seen label)

Horizons: Survival function evaluated at 1, 7, 14, 30, 45 days

Purpose: Models time-to-next-observation, capturing "burstiness" and irregular patterns

8.5 Weighted Ensemble

Final ensemble probabilities are computed as:

```
ensemble = 0.30 × rule_logistic + 0.25 × GBT + 0.25 × Weibull + 0.20 × exponential
```

Applied for each forecast horizon (1d, 7d, 14d, 30d, 45d).

-

9. Confidence Labels

Ensemble probabilities are classified into three confidence tiers:

Label	Condition
Highly likely	<code>`ensemble_prob >= 0.60` AND `freq >= 2`</code>
Possibly active	<code>`ensemble_prob >= 0.07` AND `freq >= 1`</code>
Low confidence	All other cases

Probabilities are formatted as percentage strings (e.g., "73%") in the output table.

-

10. Production Output Table

The final output table contains one row per indicator per OpDiv and includes:

Column	Description
Indicator	The threat indicator value
OpDiv	The observing organization division
Last Seen (days ago)	Days since most recent observation
Probability: 1-Day	Ensemble forecast for next-day observation

Column	Description
Probability: 7-Day	Ensemble forecast for 7-day window
Probability: 14-Day	Ensemble forecast for 14-day window
Probability: 30-Day	Ensemble forecast for 30-day window
Probability: 45-Day	Ensemble forecast for 45-day window
Confidence	Categorical label: Highly likely / Possibly active / Low confidence
Seen Today	Whether the indicator was observed on the most recent date in data

> **Known display note:** The `ensemble_45d` column may appear with its raw column name rather than the formatted "Probability: 45-Day" label in some runs, due to a rename mapping gap in `build_production_output`. This does not affect the underlying data.

-

11. How to Run

11.1 Run the Python script

Use the path your environment actually executes (scheduled task on **F.R.E.D**, or the appendix file for engineering validation):

```
py "H:\HTOC\documentation\SOP\Appendix Scripts\NextObservedIndicatorV3.0_standalone.py"
```

When a copy is published under ``Documents/HTOC Data Analytics/Python Scripts/`` on SharePoint, prefer that path after you sync it locally for runs.

11.2 Verify prerequisites

- [] [Z:\HTOC\Data Analytics\](#) is accessible and mapped.
- [] Daily OpDiv CSV files are present for at least the past 100 days in [Z:\HTOC\Data Analytics\Data\OpDiv Observations\](#).
- [] `scikit-learn` and `lifelines` are installed in the same Python environment you use to launch the script (`pip install scikit-learn lifelines`).

11.3 Monitor progress

Expect console output aligned with **Section 5** (imports, date window, CSV load, panel build, `main()` per OpDiv). If execution raises an exception, stop, fix the root cause, then start a **new** process — do not rely on partial in-memory state.

11.4 Enable file output (optional)

CSV output is currently commented out inside `main()`. To save per-OpDiv prediction files, uncomment these lines:

```
opdiv_output_dir = r'C:\Users\jaskew\Documents\NOI Logs\Predictions'  
os.makedirs(opdiv_output_dir, exist_ok=True)  
# ... save CSV block
```

Ensure the target directory exists or can be created.

11.5 Typical run time

Phase	Approximate Duration
CSV load (100 days of files)	1–3 minutes
Panel construction (Cartesian)	2–5 minutes (depends on number of OpDivs and indicators)

Phase	Approximate Duration
Feature engineering + modeling	3–10 minutes
Output formatting	< 30 seconds
Total	**~6–18 minutes**

11.6 Scheduled execution

This pipeline normally runs **automatically every day at 7:00 AM** via **Windows Task Scheduler** on **F.R.E.D.** Manual execution (**Sections 11.1–11.4**) applies when troubleshooting a failed run, running an ad hoc forecast, or validating a script change before publishing.

If the scheduled run fails, inspect Task Scheduler history on **F.R.E.D.**, confirm **`Z:\`** access, and verify OpDiv CSV availability for the run window.

-

12. Troubleshooting

Symptom	Likely Cause	Resolution
`src` is empty after load	No CSV files found at the path template	Verify `Z:\` drive is mapped; check date range and file naming convention
`ImportError: lifelines`	Package not installed	Run `pip install lifelines` in the Python environment used for the job
`NaN` probabilities in output	Single-class indicator (only ever seen or never seen)	Expected behavior; the model guard returns NaN and these are handled downstream

Symptom	Likely Cause	Resolution
Weibull AFT convergence warning	Sparse data for some indicators	Non-blocking; model still produces a result; consider excluding very-rare indicators
`ensemble_45d` appears as raw column name	Known `rename` gap in `build_production_output`	Cosmetic only; data is correct; rename column manually if needed for export
Very slow panel construction	Large number of OpDivs, users, and indicators	Reduce `start_date` offset (e.g., 60 days instead of 100) to shrink the panel
`label_45` interpretation unexpected	Single-index vs. window-slice bug in `extract_time_series_features`	Treat 45-day forecasts as approximate; prioritize 7/14/30-day horizons for decisions

-

13. Optional: Feedback Loop and Model Retraining

An optional tail section of the codebase contains a stub for per-OpDiv feedback and retraining. It is **not run as part of the standard pipeline** but can be used to improve model accuracy over time.

How it works:

Load forecast logs from [Logs/<OpDiv>/forecast_log.xlsx](#) — these should contain previous forecasts with actual outcomes appended.

Merge logs into a `train_master_{OpDiv}.csv` file (deduplicated by indicator + date key).

Retrain a `GradientBoostingClassifier(n_estimators=200, max_depth=3)` on the merged training master.

Save the trained model to `gbc_7d_{OpDiv}.pkl` for future use.

Notes:

Forecast logs must be manually maintained or generated by a separate logging process; the main pipeline does not write them automatically.

A commented-out `classification_report` block is included for optional hold-out validation.

Integration of saved `.pk1` models back into the main `main()` pipeline requires additional development.

-

14. Interpreting Results

Confidence	Recommended Action
Highly likely	Prioritize for active monitoring; high probability of re-observation within the forecast window
Possibly active	Include in standard monitoring queue; moderate probability warrants continued tracking
Low confidence	Indicator is rare or irregular; monitor passively; re-evaluate if observed again

Use the **1-day probability** for immediate triage decisions. Use **7-day and 30-day probabilities** for workload planning and longer-horizon threat tracking.

High burstiness combined with a high probability score indicates an indicator that appears in clusters — pay attention to the "Seen Today" flag for these, as a fresh observation may signal an active campaign.

15. Maintenance Notes

No persistent model files are produced by the standard run. Models are trained in-memory each execution on the current 100-day window. This ensures forecasts always reflect recent behavior.

Extending horizons: Adding a new forecast horizon (e.g., 60 days) requires adding a `label_60` in `extract_time_series_features`, new model calls in `get_model_outputs`, updated ensemble weights, and a new `classify_window` call in `add_confidence_and_format`.

Ensemble weight tuning: Weights ([0.30 / 0.25 / 0.25 / 0.20](#)) are defined in `add_rule_and_ensemble`. Adjustments should be validated against historical forecast accuracy before deploying to production.

1-day forecast note: The 1-day logistic and GBT models are currently trained on the 7-day label (`y_7`), not a dedicated 1-day label. This is a known limitation — interpret 1-day ensemble probabilities with the understanding that the supervised component reflects 7-day patterns, partially corrected by the exponential and Weibull components.

16. Appendix - Standalone Python Script

The maintained runnable script is at:

H:\HTOC\documentation\SOP\Appendix_Scripts\NextObservedIndicatorV3.0_standalone.py

Run with:

```
&"C:\Program Files\Python313\python.exe" "H:\HTOC\documentation\SOP\Appendix  
Scripts\NextObservedIndicatorV3.0_standalone.py"
```

17. Related Documents

HTOC OpDiv Observation Feed runbook

SOP_PRISM_ThreatAssessmentScoring.md — companion SOP for indicator risk scoring

SOP_Daily_Reports_Consolidation.md / SOP_Next_Obs_Daily_Batch.md — downstream daily merge;
production script on SharePoint:

[NextObserved/main.py](https://hhsgov.sharepoint.com/:u:/r/sites/HTOCDataAnalytics/Shared%20Documents/HTOC%20Data%20Analytics/Python%20Scripts/NextObserved/main.py?csf=1&web=1&e=qebIUV)

Z:\HTOC\Data Analytics\Data\OpDiv Observations\ (input data location)

Earlier major versions may exist in repository history under observationEventForecasting/ for audit only — do not assume parity with the current scheduled script.